

Proyecto Base PMS Maran – Especificación + Código Inicial para Replit

Este documento define un proyecto base para crear un PMS hotelero para Maran en Replit. Incluye: - Stack sugerido. - Estructura del proyecto. - Modelos principales (código Django). - Vistas iniciales para reservas y planning de 15 días. - Lineamientos para roles y permisos. La idea es que este documento se utilice como guía para configurar un Replit con Django y pegar el código base, y a partir de allí seguir iterando y perfeccionando el sistema.

1. Stack tecnológico sugerido

Para lograr un sistema robusto, extensible y con buen manejo de usuarios y permisos, se recomienda utilizar:

- 1 Backend: Django (framework web robusto con ORM, autenticación y administración incluidos).
- 2 Base de datos: SQLite para el MVP (fácil de usar en Replit; luego migrable a PostgreSQL).
- 3 Frontend: Plantillas Django (HTML + CSS + JS simple) para las pantallas internas.
- 4 Entorno: Replit (para desarrollo inicial; luego se puede migrar a un hosting externo).

Pasos iniciales en Replit (orientativo)

- 1) Crear un nuevo Repl de tipo "Django" o uno de "Python" y luego instalar Django con:
`pip install django`
- 2) Crear un proyecto Django (si no lo crea automáticamente):
`django-admin startproject maran_pms .`
- 3) Crear una app principal para el PMS:
`python manage.py startapp core`

Más adelante pueden crearse apps adicionales: bookings, housekeeping, restaurant, spa, hospitality. Para el MVP, se puede centralizar en una app inicial (core) y luego dividir.

2. Modelo de datos principal (Django models.py)

A continuación se presenta un conjunto de modelos que cubren el mínimo necesario para operar el PMS en Maran según los criterios definidos: - Siempre asignar habitación a la reserva. - Tarifa por reserva con posibilidad de tarifa manual y descuentos. - Estados de habitación y reservas. - Registro de quién modifica tarifas y quién genera cargos.

Código sugerido para core/models.py

```
from django.db import models
from django.contrib.auth.models import User

class Hotel(models.Model):
    name = models.CharField(max_length=200)
    code = models.CharField(max_length=20, unique=True)
    city = models.CharField(max_length=100, blank=True, null=True)

    def __str__(self):
        return self.name

class UserProfile(models.Model):
```

```

ROLE_CHOICES = [
    ('RECEPTION', 'Recepción'),
    ('HOUSEKEEPING', 'Housekeeping'),
    ('MANAGEMENT', 'Gerencia'),
    ('DIRECTOR', 'Dirección'),
    ('RESTAURANT', 'Restaurant'),
    ('SPA', 'Spa'),
    ('SECURITY', 'Seguridad'),
]
user = models.OneToOneField(User, on_delete=models.CASCADE)
hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
role = models.CharField(max_length=20, choices=ROLE_CHOICES)

def __str__(self):
    return f"{self.user.username} ({self.role})"

class RoomType(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    code = models.CharField(max_length=20)
    name = models.CharField(max_length=100)
    base_occupancy = models.IntegerField(default=2)
    max_occupancy = models.IntegerField(default=2)
    description = models.TextField(blank=True, null=True)

    class Meta:
        unique_together = ('hotel', 'code')

    def __str__(self):
        return f"{self.hotel.code} - {self.code}"

class Room(models.Model):
    STATUS_CHOICES = [
        ('AVAILABLE', 'Disponible'),
        ('OCCUPIED', 'Ocupada'),
        ('DIRTY', 'Dirty'),
        ('MAINTENANCE', 'Mantenimiento'),
        ('OOS', 'Fuera de servicio'),
    ]
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    number = models.CharField(max_length=10)
    room_type = models.ForeignKey(RoomType, on_delete=models.PROTECT)
    floor = models.CharField(max_length=10, blank=True, null=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='AVAILABLE')

    class Meta:
        unique_together = ('hotel', 'number')

    def __str__(self):
        return f"{self.hotel.code} - {self.number}"

class RatePlan(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    room_type = models.ForeignKey(RoomType, on_delete=models.PROTECT)
    base_rate = models.DecimalField(max_digits=12, decimal_places=2)
    currency = models.CharField(max_length=10, default='ARS')
    refundable = models.BooleanField(default=True)
    cancellation_policy = models.TextField(blank=True, null=True)

    def __str__(self):
        return f"{self.name} ({self.room_type.code})"

class Guest(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)

```

```

email = models.EmailField(blank=True, null=True)
phone = models.CharField(max_length=50, blank=True, null=True)
document_number = models.CharField(max_length=50, blank=True, null=True)
country = models.CharField(max_length=50, blank=True, null=True)

def __str__(self):
    return f"{self.last_name}, {self.first_name}"

class Reservation(models.Model):
    STATUS_CHOICES = [
        ('TENTATIVE', 'Tentativa'),
        ('CONFIRMED', 'Confirmada'),
        ('CHECKED_IN', 'Checked in'),
        ('CHECKED_OUT', 'Checked out'),
        ('CANCELLED', 'Cancelada'),
    ]
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    reservation_code = models.CharField(max_length=30, unique=True)
    guest = models.ForeignKey(Guest, on_delete=models.PROTECT, related_name='reservations')
    room_type = models.ForeignKey(RoomType, on_delete=models.PROTECT)
    room = models.ForeignKey(Room, on_delete=models.PROTECT)
    checkin_date = models.DateField()
    checkout_date = models.DateField()
    nights = models.IntegerField()
    rate_plan = models.ForeignKey(RatePlan, on_delete=models.SET_NULL, null=True, blank=True)
    base_rate_per_night = models.DecimalField(max_digits=12, decimal_places=2)
    discount_type = models.CharField(max_length=10, choices=[('NONE', 'None'), ('PERCENT', '%'), ('FIXED', 'Fixed')])
    discount_value = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    final_rate_per_night = models.DecimalField(max_digits=12, decimal_places=2)
    total_room_amount = models.DecimalField(max_digits=12, decimal_places=2)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='CONFIRMED')
    source = models.CharField(max_length=50, blank=True, null=True)
    notes = models.TextField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    last_modified_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)

    def __str__(self):
        return f"{self.reservation_code} - {self.guest}"

class CancelledReservationLog(models.Model):
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    reservation_code = models.CharField(max_length=30)
    guest_name = models.CharField(max_length=200)
    room_number = models.CharField(max_length=20)
    checkin_date = models.DateField()
    checkout_date = models.DateField()
    cancellation_date = models.DateTimeField(auto_now_add=True)
    cancelled_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)
    reason = models.TextField(blank=True, null=True)

class Meta:
    ordering = ['-cancellation_date']

class Charge(models.Model):
    CATEGORY_CHOICES = [
        ('ROOM', 'Room'),
        ('RESTAURANT', 'Restaurant'),
        ('SPA', 'Spa'),
        ('OTHER', 'Other'),
    ]
    hotel = models.ForeignKey(Hotel, on_delete=models.CASCADE)
    reservation = models.ForeignKey(Reservation, on_delete=models.CASCADE, related_name='charges')
    description = models.CharField(max_length=200)
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    date = models.DateField()
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES, default='ROOM')
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)

```

```
def __str__(self):  
    return f"{self.reservation.reservation_code} - {self.description}"
```

3. Reglas de negocio clave incorporadas en el diseño

- 1 Siempre se asigna una habitación a la reserva (Reservation.room es obligatorio).
- 2 Las reservas tentativas existen y afectan disponibilidad (status TENTATIVE).
- 3 No se permite overbooking: se debe validar que no haya reservas en conflicto de fechas para la misma habitación.
- 4 Las cancelaciones y borrados se registran en CancelledReservationLog para reporte diario.
- 5 Tarifa por reserva: se almacena base_rate_per_night, final_rate_per_night y total_room_amount.
- 6 Se permite tarifa manual: el usuario puede editar base_rate_per_night y final_rate_per_night según permisos.
- 7 Se guardan descuentos (tipo y valor) y quién modificó la reserva (last_modified_by).
- 8 Estados de habitación permiten: Disponible, Ocupada, Dirty, Mantenimiento, Fuera de servicio.
- 9 Check-out deja la habitación en estado Dirty (luego housekeeping la pasa a Clean/Disponible).

4. Planning de 15 días – Concepto y vista base

El Planning (Room Rack) debe mostrar un rango de 15 días, comenzando siempre en la fecha de hoy (en el MVP). Filas = habitaciones. Columnas = días. Cada celda representa una combinación (habitación, fecha) con estado visual: - Libre - Ocupada (reserva confirmada o checked in) - Tentativa - Dirty - Mantenimiento / OOS Además, debe permitir crear reservas haciendo clic en una celda Libre.

Vista Django orientativa (core/views.py)

```
from datetime import date, timedelta
from django.shortcuts import render
from .models import Room, Reservation

def planning_view(request):
    hotel = request.user.userprofile.hotel # suposición: usuario ligado a hotel
    start_date = date.today()
    days = [start_date + timedelta(days=i) for i in range(15)]

    rooms = Room.objects.filter(hotel=hotel).order_by('number')

    end_date = days[-1] + timedelta(days=1)
    reservations = Reservation.objects.filter(
        hotel=hotel,
        checkin_date__lt=end_date,
        checkout_date__gt=start_date
    ).select_related('room', 'guest')

    occupancy = {room.id: {} for room in rooms}

    for res in reservations:
        current = res.checkin_date
        while current < res.checkout_date:
            if start_date <= current <= days[-1]:
                key = current.strftime('%Y-%m-%d')
                occupancy[res.room.id][key] = res
            current += timedelta(days=1)

    context = {
        'rooms': rooms,
        'days': days,
        'occupancy': occupancy,
    }
    return render(request, 'core/planning.html', context)
```

6. Dashboard diario básico (orientativo)

```

from datetime import date
from django.shortcuts import render
from .models import Room, Reservation

def dashboard_view(request):
    hotel = request.user.userprofile.hotel
    today = date.today()

    rooms = Room.objects.filter(hotel=hotel)
    total_rooms = rooms.count()
    todays_reservations = Reservation.objects.filter(
        hotel=hotel,
        status__in=['CONFIRMED', 'CHECKED_IN'],
        checkin_date__lte=today,
        checkout_date__gt=today
    )
    occupied_rooms = todays_reservations.values_list('room_id', flat=True).distinct().count()

    arrivals = Reservation.objects.filter(hotel=hotel, checkin_date=today, status__in=['TENTATIVE', 'CONFIRMED'])
    departures = Reservation.objects.filter(hotel=hotel, checkout_date=today, status='CHECKED_IN')

    context = {
        'total_rooms': total_rooms,
        'occupied_rooms': occupied_rooms,
        'today': today,
        'arrivals': arrivals,
        'departures': departures,
    }
    return render(request, 'core/dashboard.html', context)

```

Con este proyecto base se puede: - Configurar el entorno en Replit. - Crear el proyecto Django maran_pms y la app core. - Pegar estos modelos en core/models.py. - Ejecutar migraciones (python manage.py makemigrations && python manage.py migrate). - Crear un superusuario (python manage.py createsuperuser). - Empezar a construir vistas, templates y permisos apoyándose en este diseño. A partir de aquí se pueden agregar módulos específicos para restaurant, spa, housekeeping y hospitalidad.